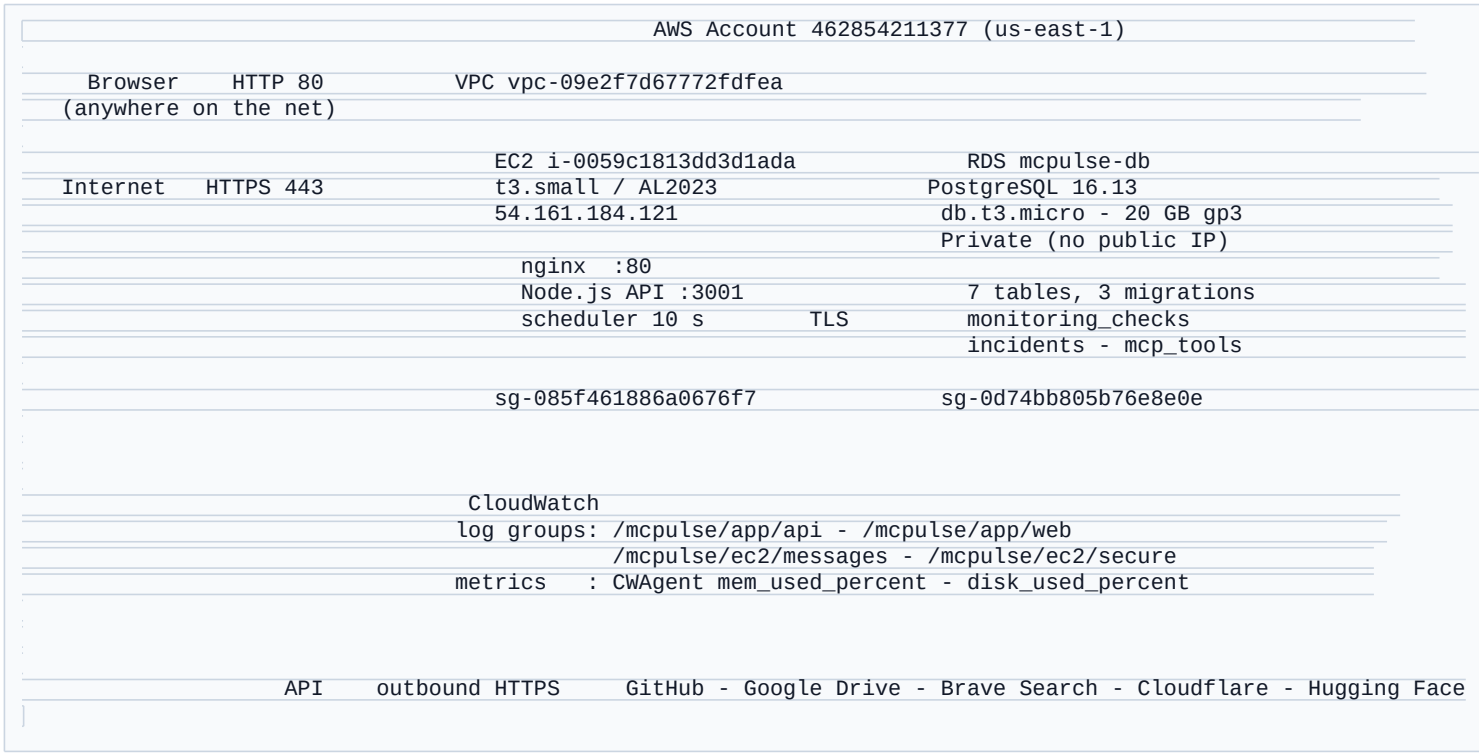
MCPulse on AWS - Architecture, Services and Data Flow

Purpose: everything you need for the "show me your AWS deployment" part of the presentation. Every service, every identifier, where the data goes and why each piece exists. All values below were read from the live deployment on **2026-10-22**.

---

1. The one-picture summary



---

2. Every service, and exactly why it is there

#	Service	Live identifier	What it does in MCPulse	Why this service (and not a local one)
1	**Amazon VPC** (default)	`vpc-09e2f7d67772fdfea`	Isolates the EC2 instance and the database. 6 subnets across `us-east-1a…f`, one Internet Gateway.	Network isolation is the core of a cloud deployment: the database is unreachable from the internet because it has no route or public address outside the VPC.
2	**Amazon EC2**	`i-08.9ma18.13oA12023da` - `54.161.184.121` - AZ `us-east-1b`	Runs the whole application as containers: nginx (React dashboard + reverse proxy) and the Node.js/Express API with the 10-second monitoring scheduler.	The API is a long-running stateful service (scheduler, in-memory locks). EC2 gives OS control, a public address and Docker. `t3.small` was chosen because `t3.micro` (1 GB) cannot build the images.
3	**Amazon RDS for PostgreSQL**	`mcpulse-db` - PostgreSQL 16.13 - `db.t3.micro` - 20 GB gp3 - `mcpulse-db.cyinxjr3lgjc.us-east-1.rds.ama`	Stores everything: monitored servers, every check with its latency and status code, incidents and covered tools, test profiles and the migration history.	Managed relational database with automated patching, backups and durability. `PubliclyAccessible = false`, so it is only reachable from inside the VPC.
4	**Security group - web tier**	`mcpulse-web-sg` - `sg-085f461886a0676f7`	Inbound: HTTP 80 + HTTPS 443 from `0.0.0.0/0` (the dashboard is public), SSH 22 from the single administrator address only.	The virtual firewall of the VPC. Port 3001 (the raw API) is **not** opened, so the API is only reachable through nginx.
5	**Security group - database tier**	`mcpulse-db-sg` - `sg-0d74bb805b76e8e0e`	Inbound: PostgreSQL 5432 **from the web security group only**.	Traffic to the database is limited to the application tier. Even a mistake elsewhere in the VPC cannot reach the database.
6	**Amazon CloudWatch Logs**	`/mcpulse/app/api`, `/mcpulse/app/web`, `/mcpulse/ec2/messages` (7-day retention), `/mcpulse/ec2/secure`	Collects the application output and the host system log.	Central place to look for what the app did, with retention that bounds the cost.
7	**Amazon CloudWatch Metrics**	namespace `CWAgent` -> `mem_used_percent`, `disk_used_percent`	Host memory and disk utilisation of the EC2 instance.	Shows that the platform is observable: capacity trends without SSH-ing into the box.

#	Service	Live identifier	What it does in MCPulse	Why this service (and not a local one)
8	**IAM role `LabRole` + `LabInstanceProfile`**	`arn:aws:iam::462854211377:instance-profile/LabInstanceProfile`	*Present in the account but **not attachable*** - the lab policy does not grant `iam:PassRole`, so no role can be bound to the instance. The CloudWatch agent therefore uses the temporary Learner Lab credentials stored in `(mode 600)` on the instance.	The intended "no static keys on the instance" pattern. It is blocked by the lab policy, which is a realistic finding to present.
9	**Amazon S3**	*not used*	The CSV telemetry export is generated directly from the database.	Creating a bucket would add cost and a second copy of data for no requirement. Deliberately skipped and documented.
10	**Key pair `mcpulse-lab`**	public part in EC2, private key in `(0600)`	SSH access to the instance for deployment and administration.	Standard, key-pair access instead of a password. The private key never leaves the operator's machine.

Cost of the whole deployment: ≈ $0.18 for a 4-hour lab session, ≈ $33 for a full month running continuously (EC2 $0.021/h, RDS $0.017/h, 20 GB gp3 $2.30/mo, public IPv4 $0.005/h). No load balancer, no NAT gateway, no Multi-AZ, no S3.

---

3. How the data actually flows

3.1 A monitoring cycle (the heart of the product)

```
every 10 s  node-cron scheduler

        reads the monitored servers from RDS        <- SELECT * FROM mcp_servers
        picks the servers whose interval is due     <- no overlap: in-memory lock per server

        for each server       HTTPS POST to the provider endpoint
                              JSON-RPC 2.0  "initialize"
                              protocol version 2025-03-26
                              Accept: application/json, text/event-stream



            JSON response                    SSE response
            (text/event-stream handled by the checker as well)

                      measured round-trip latency  -> duration_ms
                      HTTP status code             -> status_code
                      error category (sanitised)  -> error_code / sanitized_error


                    INSERT INTO monitoring_checks (...)      <- every check is persisted

                                        3 consecutive failures -> INSERT INTO incidents (state='open')
                                        first success        -> UPDATE incidents SET state='resolved'
```

Live proof from the database (538 checks, 5 incidents, 4 tools, 3 migrations):

```
 provider                      outcome    latency   http
 Brave Search MCP              failure     205 ms   403
 Google Drive Remote MCP       success      38 ms   200
 GitHub Remote MCP             failure      90 ms   401
 Hugging Face Hub              success      36 ms   200
 Cloudflare Official MCP       failure      25 ms   401
```

3.2 A user request

```
browser   HTTP    nginx (EC2, host networking, port 80)
                        /            -> static React bundle
                        /api/…       -> proxy_pass 127.0.0.1:3001

                Express API (container)
                        auth: state changing requests need X-MCPulse-Admin-Key
                        SSRF guard: private / loopback / metadata targets are rejected
                        TLS to the database: chain + hostname verified against the RDS CA bundle

                PostgreSQL 16 (RDS, private, port 5432)
```

3.3 Where each AWS service is touched

```
 request                 EC2 / nginx       CloudWatch Logs  (app + system output)
                         + API             CloudWatch Metrics (mem, disk)

                                SQL over TLS :5432 (security-group to security-group)


                         RDS Postgres       persistent storage (gp3 volume)


        outbound HTTPS 443
        (scheduled checks)
        to the 5 providers       (the API reaches them through the EC2 public IP)
```

---

4. What to screenshot in the AWS console

Take these shots in this order - they tell the whole story.

#	Console page	What it proves
1	**EC2 -> Instances**	The instance is `running`, type `t3.small`, with the public IP and the `mcpulse-web-sg` attached.
2	**EC2 -> Security Groups** (web tier)	Rules: 80/443 open, **22 from one address only**, 3001 absent.
3	**EC2 -> Security Groups** (database tier)	Rule: 5432 **only** from the web security group.
4	**RDS -> Databases**	`mcpulse-db`, *available*, **Publicly Accessible = No**, 20 GB gp3, endpoint, `mcpulse-db-sg`.
5	**RDS -> Connectivity** (Security groups tab)	The database is attached to the database security group only.
6	**CloudWatch -> Log groups**	The four `/mcpulse/...` groups with 7-day retention.
7	**CloudWatch -> Metrics -> CWAgent**	`mem_used_percent` and `disk_used_percent` for the instance.
8	**VPC -> Security groups / Subnets**	The default VPC with 6 subnets across 6 AZs.
9	**EC2 -> Instance -> SSH in** (`docker compose ps`)	`api` healthy, `web` up - the containers actually running.
10	**Browser -> the dashboard**	The product itself, with live latency and status.

---

5. The security model in one table

Threat	Control	Evidence
Database exposed to the internet	`PubliclyAccessible = false` + a security group that only accepts 5432 from the web tier	RDS console, `describe-db-instances`
Man-in-the-middle on the database link	TLS with the certificate chain **and** the hostname verified against the Amazon RDS CA bundle	RDS refuses unencrypted connections: *"no pg_hba.conf entry … no encryption"*
Raw API reachable from the internet	The API port is published on `127.0.0.1:3001` only; nginx is the single public entry point	`docker compose ps` shows `127.0.0.1:3001->3001/tcp`
Anyone changing the monitored configuration	Every state changing request must carry the administrative key	`POST /api/servers` without the key -> **401**
SSRF against the cloud metadata service	The guard rejects loopback, private, link-local and metadata addresses before any outbound request	`POST` with `http://169.254.169.254/…` -> **400**
SSH brute force	Port 22 restricted to the single administrator address	Security group rule
Secrets in the repository	The database password, the SSH key and the AWS credentials live in `~/.mcpulse/` and `~/.aws/`, never in Git	secret scan of the tree: clean
Log cost runaway	7-day retention on every log group	`describe-log-groups`

---

6. Honest limitations (present these; they look professional, not defensive)

1. **No instance profile.** The lab policy denies `iam:PassRole`, so `LabRole` cannot be attached. The CloudWatch agent uses the temporary Learner Lab credentials on the instance, and delivery stops when the lab session ends.
2. **CloudWatch log events and metric datapoints do not arrive** in this account, although the agent runs, reads the files and creates the log groups and the metric definitions. `aws logs put-log-events` from the instance succeeds with the same credentials, so the API path is healthy - the gap is inside the agent's delivery loop. The application's own data is unaffected and is fully visible in the database and in the dashboard.
3. **No custom DB subnet group.** The account rejects the default VPC subnets for `CreateDBSubnetGroup`, so the instance uses the default subnet group. The database is still private (no public address, security-group restricted).
4. **Ubuntu is not available** to this role, so the instance runs Amazon Linux 2023.
5. **Three of the five providers need credentials** (GitHub 401, Cloudflare 401, Brave 403). MCPulse reports them as `Down` with a provider-specific explanation - that is correct monitoring behaviour, not a defect. Per-provider credentials would need a schema change.
6. **The administrative key is compiled into the dashboard bundle**, so write operations are protected against casual use rather than against a determined attacker. Build without `VITE_ADMIN_KEY` for a stricter posture.
7. **Not a Git repository**, so the deployment transfers sources with `tar` over SSH.

---

7. Cost and lab etiquette

Item	Rate	4-hour session
EC2 `t3.small`	$0.021 / h	$0.08
RDS `db.t3.micro`	$0.017 / h	$0.07
20 GB gp3	$2.30 / month	$0.004
Public IPv4	$0.005 / h	$0.02
Total		≈ **$0.18**

`scripts/aws/stop.sh` stops the instance and the database without deleting anything, so the environment can be resumed the next day with `scripts/aws/resume.sh`.